

17.模型剪枝

1. 模型剪枝介绍

模型剪枝一般可分为有损剪枝和无损剪枝两种方式，RKNN-Toolkit2的模型剪枝为**无损剪枝**，也就是可以在减小模型大小和计算量的前提下，不会降低模型的浮点精度，甚至可以提高模型的量化精度。

但并不是所有模型都可以进行无损剪枝，无损剪枝是根据模型的权重的稀疏化程度，来去除一些对模型结果不造成影响的权重和Feature通道，以减小模型的大小和模型的计算量。在启用模型剪枝配置后（将`rknn.config()` 的 `model_pruning` 参数设为 `True`），模型转换时会自动根据权重的稀疏化程度对模型进行剪枝。

如果模型剪枝成功，则会打印剪枝结果，如下：

代码块

```
1 I model_pruning results:
2 I Weight: -1.12145 MB (-6.9%)
3 I GFLOPs: -0.15563 (-13.4%)
```

其中，Weight剪掉了1.12145 MB（占比6.9%），模型运算量减少0.15563 GFLOPs（占比13.4%）。

注：如果因模型稀疏化程度不够而剪枝失败，则不会做任何处理（也不会打印上述信息），因此该配置并不

会影响正常的模型转换。

结构化剪枝 + 微调

这是目前边缘设备部署中最常用、最稳妥的方法。与单纯依赖转换工具的自动剪枝不同，它需要在PyTorch 训练阶段主动介入。

- 核心逻辑：通过移除整个卷积通道或网络层（即“结构化剪枝”），让剪枝后的模型依然保持规整的结构。这样 NPU 才能利用 DMA 连续读取和编译器优化实现真正的加速。
- 标准流程：通常遵循“预训练基线模型 → 使用第三方库（如 `torch-pruning`）执行结构化剪枝 → 微调恢复精度 → 再结合 INT8 量化”的完整链路。通过这种闭环操作，可以将 YOLOv8 等模型参数量缩减 30% 以上，推理速度提升 2-3 倍。

2. API介绍

2.1 模型配置

API	config
	remove_weight: 去除conv等权重以生成一个RKNN的从模型，该从模型可以与带完整权重的RKNN模型共享权重以减少内存消耗。默认值为False。
	compress_weight: 压缩模型权重，可以减小RKNN模型的大小。默认值为False。
	single_core_mode: 是否仅生成单核模型，可以减小RKNN模型的大小和内存消耗。默认值为False。目前仅对RK3588 / RK3576生效。默认值为False。
	model_pruning: 对模型进行无损剪枝。对于权重稀疏的模型，可以减小转换后RKNN模型的大小和计算量。默认值为False。
	op_target: 用于指定OP的具体执行目标（可以设置运行在NPU/CPU/GPU等），通过输出tensor名的方式，{'op0_output_name':'cpu','op1_output_name':'npu',...}，或者通过指定op_type的方式，{'op_type':'cpu'....}，同时还可以支持混合使用，{'op_type':'cpu','op0_output_name':'cpu'....}。默认值为None。 其中，'op0_output_name'和'op1_output_name'为对应OP的输出tensor名，可以通过精度分析（accuracy_analysis）功能的返回结果中获取。'cpu'和'npu'则表示该tensor对应的OP的执行目标是CPU或NPU，目前可选的选项有：'cpu'/'npu'/'gpu'/'auto'，其中，'auto'是自动选择执行目标。指定op_type的方式，同样需要通过精度分析返回结果中获取，可以将模型内所有该op_type的执行目标设置到CPU或NPU，例如，{'Add':'cpu','Expand':'cpu',....}。
	dynamic_input: 用于根据用户指定的多组输入shape，来模拟动态输入的功能。格式为[[input0_shapeA, input1_shapeA, ...], [input0_shapeB, input1_shapeB, ...], ...]。 默认值为None，实验性功能。 假设原始模型只有一个输入，shape为[1,3,224,224]，或者原始模型的输入shape本身就是动态的，如shape为[1,3,height,width]或[1,3,-1,-1]，但部署的时候，需要该模型支持3种不同的输入shape，如[1,3,224,224], [1,3,192,192]和[1,3,160,160]，此时可以设置dynamic_input=[[1,3,224,224]], [[1,3,192,192]], [[1,3,160,160]]，转换成RKNN模型后进行推理时，需传入对应shape的输入数据。 注： 1. 需要原始模型本身支持动态输入才可开启此功能，否则会报错。

2.2 加载caffe模型

API	load_caffe
描述	加载Caffe模型。（ARM64版本暂不支持该接口）
参数	model: Caffe模型文件（.prototxt后缀文件）路径。
	blobs: Caffe模型的二进制数据文件（.caffemodel后缀文件）路径。
	input_name: Caffe模型存在多输入时，可以通过该参数指定输入层名的顺序，形如['input1','input2','input3']，注意名字需要与模型输入名一致；默认值为None，表示按Caffe模型文件（.prototxt后缀文件）自动给定。
返回值	0: 导入成功。
	-1: 导入失败。

3. 剪枝代码测试

代码块

```

1  import numpy as np
2  import cv2
3  from rknn.api import RKNN
4
5  def show_outputs(outputs):
6      # 将输出结果展平成一维数组
7      output = outputs[0].reshape(-1)
8      # 对输出结果的索引进行排序，按对应值从大到小排列（获取置信度最高的预测）
9      index = sorted(range(len(output)), key=lambda k : output[k], reverse=True)
10     # 打开标签文件，读取类别名称
11     fp = open('./labels.txt', 'r')
12     labels = fp.readlines()
13     # 初始化打印字符串，显示模型名称和TOP 5标题
14     top5_str = 'mobilenet\n-----TOP 5-----\n'
15     for i in range(5):
16         value = output[index[i]]
17         if value > 0:
18             # 格式化输出：索引、得分、类别名称
19             # 假设labels.txt中每行格式为 "index:className", 故用split(':')[1]获取
20             topi = '[{:>3d}] score:{:.6f} class:"{}"\n'.format(index[i],
21 value, labels[index[i]].strip().split(':')[1])
22         else:
23             # 如果得分不大于0，显示无效结果
24             topi = '[-1]: 0.0\n'
25     top5_str += topi

```

```

25     # 打印最终的TOP 5预测结果
26     print(top5_str.strip())
27
28 if __name__ == '__main__':
29
30     # 创建RKNN对象, verbose=True表示开启详细日志输出
31     rknn = RKNN(verbose=True)
32
33     # 配置模型的预处理参数
34     print('--> Config model')
35     rknn.config(mean_values=[103.94, 116.78, 123.68], # 减去均值(R,G,B), 用于数
数据归一化
36                  std_values=[58.82, 58.82, 58.82], # 除以标准差(R,G,B), 用于数
数据归一化
37                  quant_img_RGB2BGR=True, # 量化时将图像从RGB转换为BGR
(OpenCV默认格式)
38                  target_platform='rk3588', # 指定目标平台为RK3588
39                  model_pruning=True) # 开启模型剪枝优化
40     print('done')
41
42     # 加载Caffe模型
43     # 来源参考: https://github.com/shicai/MobileNet-Caffe
44     print('--> Loading model')
45     ret = rknn.load_caffe(model='./mobilenet_deploy.prototxt', # 网络结构文件
46                          blobs='./mobilenet.caffemodel') # 模型权重文件
47     if ret != 0:
48         print('Load model failed!')
49         exit(ret)
50     print('done')
51
52     # 构建模型 (进行量化和转换)
53     print('--> Building model')
54     # do_quantization=True 表示进行量化 (将浮点数转换为整数以加速推理)
55     # dataset.txt 包含用于量化的校准数据集路径列表
56     ret = rknn.build(do_quantization=True, dataset='./dataset.txt')
57     if ret != 0:
58         print('Build model failed!')
59         exit(ret)
60     print('done')
61
62     # 导出RKNN模型文件
63     print('--> Export rknn model')
64     ret = rknn.export_rknn('./mobilenet.rknn')
65     if ret != 0:
66         print('Export rknn model failed!')
67         exit(ret)
68     print('done')

```

